



# 关系模型和关系代数

## 关系模型

### 超码 (Superkey)

设  $K \subseteq R$ , 如果在关系模式  $R$  中,  $K$  的取值足以唯一标识关系  $r(R)$  中的每一个元组, 那么  $K$  就是一个**超码**。

### 候选码 (Candidate key)

若一个超码是**最小的**, 即去掉其中任何一个属性后就不再是超码, 那么它就是一个**候选码**。

### 主码 (Primary key)

在多个候选码中, 由数据库设计者选定一个作为标识关系中元组的**主码**。

主码用于唯一标识关系中的每一行记录。

### 外码 (Foreign Key)

在一个关系模式  $R_1$  中, 可能包含另一个关系模式  $R_2$  的主码作为其属性之一。这个属性称为**从  $R_1$  指向  $R_2$  的外码**。

- 在这种外码依赖关系中,
  - $r_1$  被称为**参照关系 (referencing relation)**,
  - $r_2$  被称为**被参照关系 (referenced relation)**。

### 参照完整性约束 (Referential Integrity Constraint)

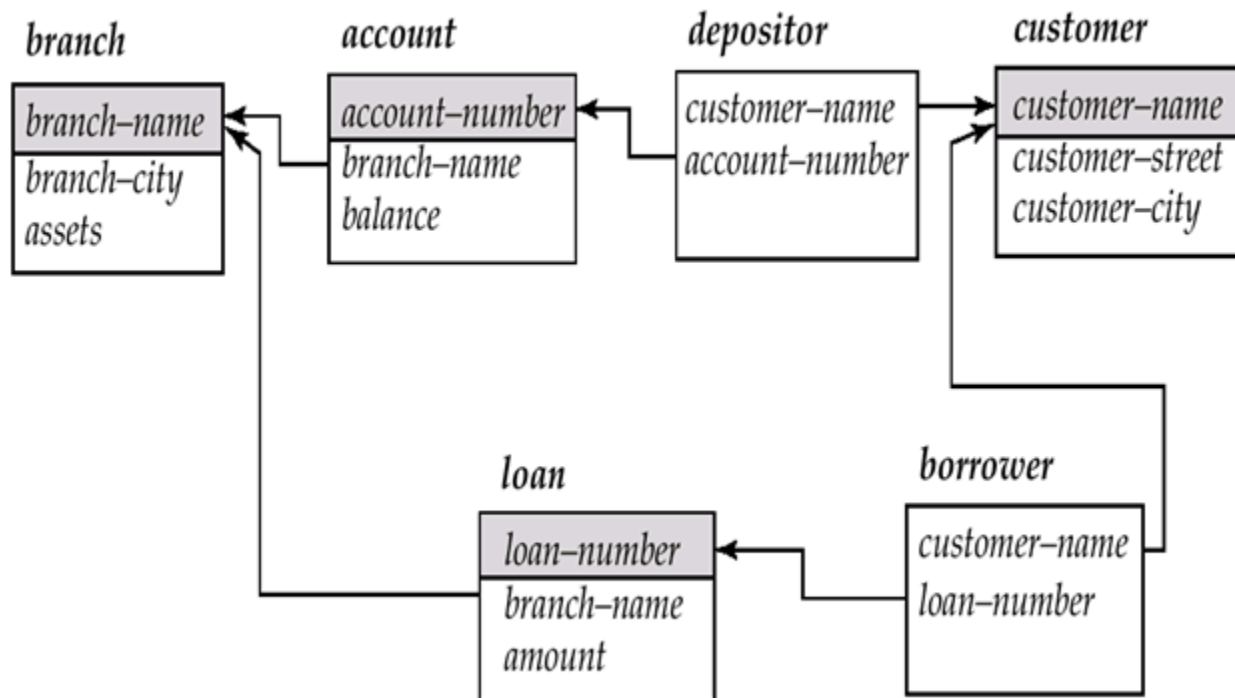
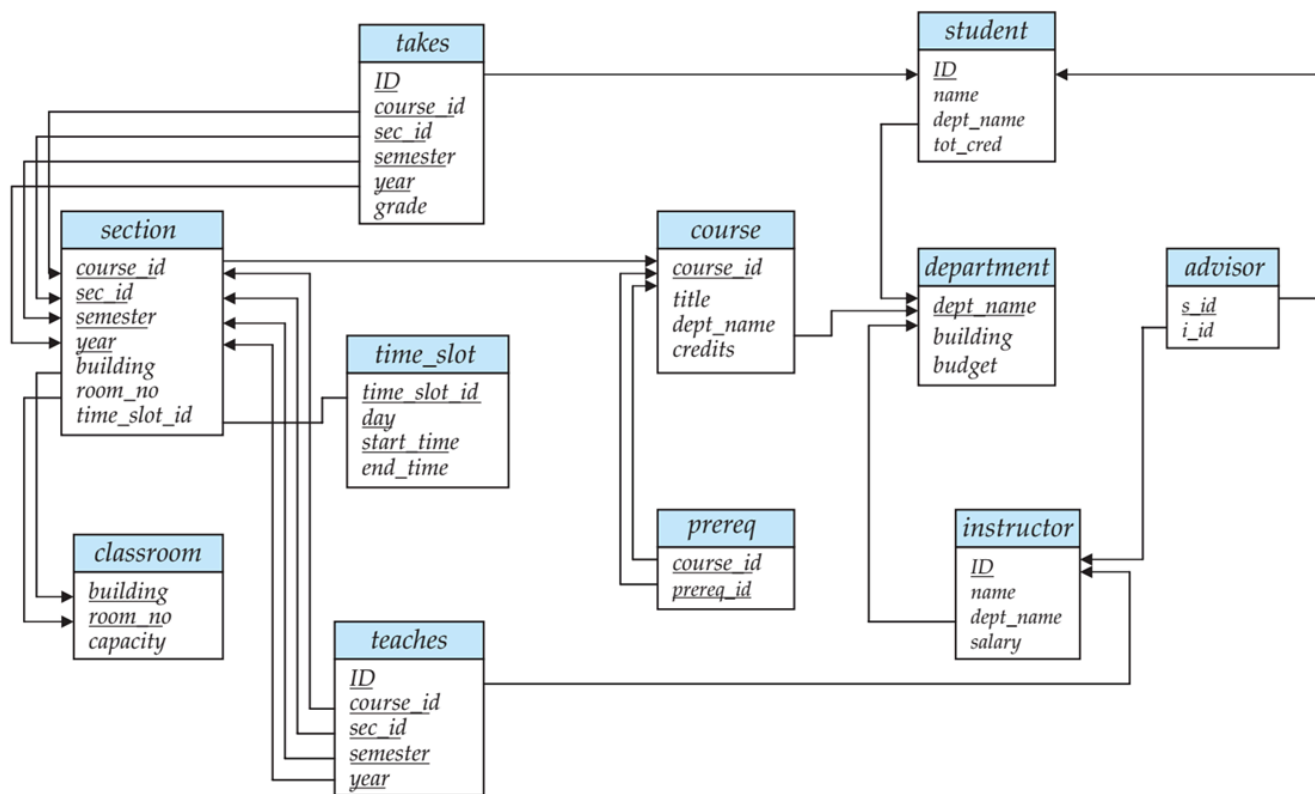
参照关系中任一元组的外码属性, 其取值必须出现在被参照关系中某个元组的对应主码属性中。

即: **外码的值必须在被参照关系中存在对应值**。

### 模式图 (Schema Diagram)

一个数据库模式连同其主码和外码的依赖关系, 可以通过**模式图** (schema diagram) 以图形方式表示。

模式图通常用于展示各个表之间的逻辑联系。



# 关系代数

**定义**一种过程式语言，由一组操作组成，每个操作以一个或两个关系作为输入，并产生一个新

的关系作为结果。

## 选择操作

$$\sigma_P(r) = \{t \mid t \in r \text{ 且 } P(t)\}$$

其中,  $P$  是**选择谓词 (selection predicate)**, 由以下运算符组成:

- 逻辑运算:  $\wedge$  (and)、 $\vee$  (or)、 $\neg$  (not)
- 比较运算:  $=$ 、 $\neq$ 、 $<$ 、 $>$ 、 $\leq$ 、 $\geq$

选出instructor关系中dept\_name为“Physics”的所有元组。

$$\sigma_{\text{dept\_name}=\text{"Physics"}}(\text{instructor})$$

可以使用逻辑连接词 ( $\wedge$ 、 $\vee$ 、 $\neg$ ) 将多个谓词组合成更复杂的条件。  
找出属于 Physics 系且工资高于 90,000 的教师。

$$\sigma_{\text{dept\_name}=\text{"Physics"} \wedge \text{salary} > 90000}(\text{instructor})$$

选择谓词中可以比较**两个属性**的值。  
找出那些系名和楼名相同的系。

$$\sigma_{\text{dept\_name}=\text{building}}(\text{department})$$

## 投影操作

$$\Pi_{A_1, A_2, \dots, A_k}(r)$$

- $A_1, A_2, \dots, A_k$  是属性名,
- $r$  是关系名,
- 结果是一个只保留指定属性列的关系, **删除未列出的列**,
- 并且由于关系是集合, **重复的元组将被自动去除**。

从 instructor表中**去掉dept\_name属性**, 只保留ID、name和salary三列。

$$\Pi_{\text{ID}, \text{name}, \text{salary}}(\text{instructor})$$

## 集合并操作

$$r \cup s = \{t \mid t \in r \text{ 或 } t \in s\}$$

**使用条件：**

- **r 和 s 必须同元 (arity 相同)**：即有相同数量的属性。
- **属性域必须相容 (compatible)**：例如 r 和 s 的第 2 列都必须表示相同类型的数据（如都是课程编号、姓名等）。

找出在 **2021 年秋季** 或 **2022 年春季**（或两者）开设的所有课程。

$$\Pi_{\text{course\_id}}(\sigma_{\text{semester}=\text{"Fall"} \wedge \text{year}=2021}(\text{section})) \cup \Pi_{\text{course\_id}}(\sigma_{\text{semester}=\text{"Spring"} \wedge \text{year}=2022}(\text{section}))$$

## 集合差操作

**差运算符表示法：**

$$r - s = \{t \mid t \in r \text{ 且 } t \notin s\}$$

**使用条件：** 和集合并相同

找出在 **2021 年秋季** 开设、但**没有在 2022 年春季**开设的所有课程。

$$\Pi_{\text{course\_id}}(\sigma_{\text{semester}=\text{"Fall"} \wedge \text{year}=2021}(\text{section})) - \Pi_{\text{course\_id}}(\sigma_{\text{semester}=\text{"Spring"} \wedge \text{year}=2022}(\text{section}))$$

## 笛卡尔积操作

**笛卡尔积 (Cartesian product) 符号表示法：**

$$r \times s = \{tq \mid t \in r \text{ 且 } q \in s\}$$

即：将  $r$  中的每个元组与  $s$  中的每个元组组合，得到一个新元组。

**要求：**

- $r(R)$  和  $s(S)$  的**属性集合必须互不相交**，即

$$R \cap S = \emptyset$$

- 如果两者有**重复属性名**，则**必须重命名**属性，以避免歧义。

## 更名操作

允许我们为关系代数表达式的结果命名，从而可以引用这些结果。

例如， $\rho_X(E)$  将表达式  $E$  的结果命名为  $X$ 。

如果关系代数表达式  $E$  的参数数量为  $n$ ,

$\rho_{X(A_1, A_2, \dots, A_n)}(E)$  会返回表达式  $E$  的结果，并将其命名为  $X$ ，同时将属性重命名为  $A_1, A_2, \dots, A_n$ 。

## 集合交操作

$$r \cap s = \{t \mid t \in r \text{ 且 } t \in s\}$$

### • 前提条件：

- 关系  $r$  和  $s$  必须具有相同的**元数** (arity)
- $r$  和  $s$  的属性必须是**兼容的**

**注意：**交集运算可以表示为差集的组合形式

$$r \cap s = r - (r - s)$$

查找在2021年秋季和2022年春季学期都开设的所有课程：

$$\pi_{course\_id}(\sigma_{semester="Fall" \wedge year=2021}(section)) \cap \pi_{course\_id}(\sigma_{semester="Spring" \wedge year=2022}(section))$$

## 自然连接操作

$$r \bowtie s$$

设关系  $r$  和  $s$  的模式分别为  $R$  和  $S$ ，则  $r \bowtie s$  的结果是一个模式为  $R \cup S$  的关系，其计算过程如下：

1. 对于  $r$  中的每个元组  $t_r$  和  $s$  中的每个元组  $t_s$
2. 如果  $t_r$  和  $t_s$  在  $R \cap S$  的所有属性上取值相同，则将它们合并为一个新元组  $t$ ：
  - $t$  在  $R$  上的属性值取自  $t_r$
  - $t$  在  $S - R$  上的属性值取自  $t_s$

**示例：**

- $R = (A, B, C, D)$

- $S = (E, B, D)$
- 结果模式:  $(A, B, C, D, E)$
- 自然连接可表示为:

$$\Pi_{r.A, r.B, r.C, r.D, s.E} (\sigma_{r.B=s.B \wedge r.D=s.D} (r \times s))$$

**特殊情况:**

若  $R \cap S = \emptyset$ , 则自然连接退化为笛卡尔积:

$$r \bowtie s = r \times s$$

## θ连接操作

自然连接的扩展, 允许将选择和笛卡尔积组合为单一操作。

对于关系  $r(R)$  和  $s(S)$ , 设  $\theta$  是模式  $R \cup S$  上的谓词, 则  $\theta$  连接定义为:

$$r \bowtie_{\theta} s = \sigma_{\theta}(r \times s)$$

## 外连接操作

一种避免信息损失的连接操作扩展

计算连接后, 将未匹配的元组 (来自任一关系) 添加到连接结果中, 并使用空值:

- 空值表示该值未知或不存在
- 所有涉及空值的比较 (粗略地说) 按定义为假

**形式化定义:**

设  $r(R)$  和  $s(S)$  为关系,  $R \cap S$  为公共属性集

外连接  $r \Join s$  计算如下:

1. 首先计算自然连接  $r \bowtie s$
2. 对于  $r$  中未匹配的元组  $t_r$ , 补充  $S - R$  属性为空值后加入结果
3. 对于  $s$  中未匹配的元组  $t_s$ , 补充  $R - S$  属性为空值后加入结果

**三种变体:**

- 左外连接  $\Join_L$ : 保留左表所有元组
- 右外连接  $\Join_R$ : 保留右表所有元组
- 全外连接  $\Join_{\text{all}}$ : 保留两侧所有元组

# 除操作

$$r \div s$$

设关系 $r$ 和 $s$ 的模式分别为 $R$ 和 $S$ , 其中:

- $R = (A_1, \dots, A_m, B_1, \dots, B_n)$
- $S = (B_1, \dots, B_n)$

则 $r \div s$ 的结果是模式为 $R - S = (A_1, \dots, A_m)$ 上的关系, 定义为:

$$r \div s = \{t \mid t \in \Pi_{R-S}(r) \wedge \forall u \in s(tu \in r)\}$$

**满足条件:**

1.  $t$ 必须出现在 $\Pi_{R-S}(r)$ 中
2. 对于 $s$ 中的每个元组 $t_s$ ,  $r$ 中都存在一个元组 $t_r$ 满足:
  - $t_r[S] = t_s[S]$
  - $t_r[R - S] = t$

**基于基本代数运算的定义:**

$$r \div s = \Pi_{R-S}(r) - \Pi_{R-S}((\Pi_{R-S}(r) \times s) - \Pi_{R-S,S}(r))$$

**说明:**

- $\Pi_{R-S,S}(r)$ 只是对 $r$ 的属性进行重新排序
- $\Pi_{R-S}((\Pi_{R-S}(r) \times s) - \Pi_{R-S,S}(r))$ 得到的是那些在 $\Pi_{R-S}(r)$ 中存在某个 $u \in s$ 使得 $tu \notin r$ 的元组 $t$

Relations  $r, s$ :

| $A$        | $B$ |
|------------|-----|
| $\alpha$   | 1   |
| $\alpha$   | 2   |
| $\alpha$   | 3   |
| $\beta$    | 1   |
| $\gamma$   | 1   |
| $\delta$   | 1   |
| $\delta$   | 3   |
| $\delta$   | 4   |
| $\epsilon$ | 6   |
| $\epsilon$ | 1   |
| $\beta$    | 2   |

$r$

| $B$ |
|-----|
| 1   |
| 2   |

$s$

$r \div s$ :

| $A$      |
|----------|
| $\alpha$ |
| $\beta$  |

## 赋值操作

**赋值操作** ( $\leftarrow$ ) 提供了一种便捷的方式来表达复杂查询  
将查询编写为顺序程序，包含：

1. 一系列赋值语句
2. 最后是一个表达式，其值作为查询结果显示

赋值必须作用于临时关系变量

例如，将  $r \div s$  表示为：

$$\begin{aligned}
 temp_1 &\leftarrow \Pi_{R-S}(r) \\
 temp_2 &\leftarrow \Pi_{R-S}((temp_1 \times s) - \Pi_{R-S,S}(r)) \\
 result &= temp_1 - temp_2
 \end{aligned}$$

## 广义投影

允许在投影列表  $\Pi_{F_1, F_2, \dots, F_n}(E)$  中使用算术表达式：



- $E$  是任意关系代数表达式
- 每个  $F_i$  是由常量或  $E$  的模式中的属性构成的算术表达式

给定关系  $credit\_info(customer\_name, limit, credit\_balance)$ , 计算每位客户剩余可消费额度:

$$\Pi_{customer\_name, limit-credit\_balance}(credit\_info)$$

## 聚合函数

接收值集合并返回单一结果值:

- avg: 平均值
- min: 最小值
- max: 最大值
- sum: 求和
- count: 计数

### 关系代数中的聚合操作

$$G_1, G_2, \dots, G_n \mathcal{G}_{F_1(A_1), F_2(A_2), \dots, F_m(A_m)}(E)$$

### 参数说明

- $E$ : 任意关系代数表达式
- $G_1, G_2, \dots, G_n$ : 分组属性列表 (可为空)
- $F_i$ : 聚合函数 (如sum/avg等)
- $A_i$ : 被聚合的属性

### 特性说明

1. 聚合结果默认无属性名
2. 可通过重命名操作赋予名称
3. 为方便起见, 允许在聚合操作中直接重命名

### 示例

若需为聚合结果命名, 可采用:

$$\rho_{result\_name} \left( \begin{matrix} \\ group\_attr \end{matrix} \mathcal{G}_{sum(attr) as total}(E) \right)$$

或直接嵌入重命名：

```
dept_id G sum(salary) as dept_total(employee)
```

## 空值（NULL）处理规则

- 1. **空值语义**
  - 表示未知值或不存在的值
  - 参与算术运算时，结果始终为NULL
- 2. **聚合函数行为**
  - 自动忽略NULL值（如 `sum(NULL,1,2)=3` ）
- 3. **去重与分组**
  - 两个NULL视为相等（与SQL一致）
- 4. **三值逻辑运算**

| 运算符 | 运算规则                          |
|-----|-------------------------------|
| OR  | unknown OR true = true        |
|     | unknown OR false = unknown    |
|     | unknown OR unknown = unknown  |
| AND | true AND unknown = unknown    |
|     | false AND unknown = false     |
|     | unknown AND unknown = unknown |
| NOT | NOT unknown = unknown         |

- 5. **SQL特殊处理**
  - P IS UNKNOWN: 在P为unknown时返回true
  - WHERE子句中，unknown结果按false处理

## 删除操作

删除请求的表示方式与查询类似，但不会向用户显示元组，而是从数据库中移除选中的元组。

## 删除规则

1. **只能删除完整元组**，不能单独删除特定属性值
2. **关系代数表示**：

$$r \leftarrow r - E$$

- $r$  是目标关系
- $E$  是关系代数查询，用于选择要删除的元组

## 插入操作

向关系中插入数据时，可以：

1. **直接指定要插入的元组**
2. **通过查询生成待插入的元组集合**

### 关系代数表示

$$r \leftarrow r \cup E$$

- $r$ ：目标关系
- $E$ ：关系代数表达式（生成待插入的元组）

## 更新操作 (Update Operation)

提供修改元组中特定属性值的能力，而无需更改其他属性。

### 基于广义投影的更新方法

1. **全关系更新**：

$$r \leftarrow \Pi_{F_1, F_2, \dots, F_n}(r)$$

- 对于**不需修改**的属性： $F_i$  直接取原属性（如  $F_1 = A_1$ ）
- 对于**需修改**的属性： $F_i$  为计算新值的表达式（如  $F_2 = A_2 + 100$ ）

2. **条件更新（部分元组）**：

$$r \leftarrow \Pi_{F_1, \dots, F_n}(\sigma_P(r)) \cup (r - \sigma_P(r))$$

- $P$ ：选择待更新元组的条件
- 执行逻辑：

- 对满足 $P$ 的元组进行属性修改（通过投影）
- 保留不满足 $P$ 的原始元组
- 取并集得到最终结果

## 视图

create view  $V$  as  $\langle \text{query expression} \rangle$

### 视图特性

1. 视图定义后，名称 $V$ 可用来引用该视图生成的**虚拟关系**
2. 视图定义**不同于**通过查询表达式创建新关系：
  - 视图定义会保存表达式
  - 使用视图时，该表达式会被替换到查询中

### 视图的层级依赖与展开

1. **视图依赖关系**
  - **直接依赖**：视图 $v_1$ 直接依赖视图 $v_2$ ，当 $v_2$ 出现在 $v_1$ 的定义表达式中
  - **传递依赖**：若存在依赖路径 $v_1 \rightarrow \dots \rightarrow v_2$ ，则 $v_1$ 依赖 $v_2$
  - **递归视图**：视图 $v$ 依赖自身时称为递归

2. **多级视图的语义定义**

对于通过其他视图定义的视图 $v_1$ （其定义表达式为 $e_1$ ，可能包含其他视图），通过**视图展开**确定语义：

视图展开算法：

```
repeat
    在 $e_1$ 中查找任意视图关系 $v_i$ 
    将 $v_i$ 替换为其定义表达式
until  $e_1$ 中不再包含视图关系
```

3. **终止条件**

只要视图定义不构成递归，该循环必然终止。最终得到的 $e_1$ 为纯基础关系表达式，即视图的物化形态。